

Specifikace jazyka SOL26 (IPP 2025/2026)

Ondřej Ondryáš, Zbyněk Křivka

Obsah

1	Popis jazyka SOL26	1
1.1	Lexikální jednotky	2
1.2	Syntaxe a sémantika jazykových konstrukcí	2
1.2.1	Výrazy a selektory	3
1.2.2	Blok	3
1.2.3	Příkaz přiřazení	4
1.2.4	Proměnné a rozsah platnosti	4
1.2.5	Blok jako objekt	4
1.2.6	Výsledek bloku	5
1.2.7	Reference na příjemce zprávy self	5
1.2.8	Klíčové slovo super	6
1.2.9	Výrazy a zasílání zpráv	6
1.2.10	Třídy, instanční metody a atributy	7
1.2.11	Příklady	10
2	Vestavěné třídy SOL26	12
3	Přehled chybových návratových kódů	15
4	Popis XML reprezentace programu	16

Úvod

Tento dokument popisuje jazyk SOL26, pro který budete implementovat interpret. Jazyk je popsán v jeho čitelné podobě, tedy pomocí lidsky čitelné syntaxe a lexikálních pravidel. S využitím této syntaxe je pak vysvětlena sémantika, tj. chování jednotlivých konstrukcí jazyka.

Pozor, váš interpret na vstupu nebude dostávat programy přímo v čitelné podobě jazyka SOL26, ale převedené z této syntaxe do formátu SOL-XML, což je abstraktní syntaktický strom reprezentovaný ve formátu XML (podrobně popsán v sekci 4). Překladač SOL2XML (překládá z jazyka SOL26 do formátu SOL-XML), který můžete využít při tvorbě vlastních experimentálních programů v jazyce SOL26, je dostupný v git repozitáři k projektu.

Při implementaci a odevzdávání se nezapomeňte **řídit pokyny v hlavním zadání** a v README souborech, které jsou součástí šablony projektu v git repozitáři.

1 Popis jazyka SOL26

SOL26 je interpretovaný, imperativní, netypovaný, třídní objektově orientovaný programovací jazyk s dynamickou typovou kontrolou. Je volně inspirován vlastnostmi a syntaxí jazyka Smalltalk. Podporuje definice tříd s instančními metodami a atributy, jednoduchou třídní dědičnost a polymorfni invokaci metod. Obsahuje několik vestavěných tříd (s třídními i instančními metodami), uživatel může definovat vlastní třídy. Syntaxe jazyka (v jeho lidsky čitelné podobě) je jednoduchá, aby bylo možné jej snadno převést na abstraktní syntaktický strom (AST), který bude (ve formátu SOL-XML) hlavním vstupem interpretu ve vašem projektu.

Všechny hodnoty (tj. výsledky výrazů a hodnoty referencované v proměnných nebo instančních atributech¹) jsou objekty, tedy instance nějaké třídy. SOL26 využívá **referenční sémantiku**, což znamená, že typem proměnné je třída objektu, který je referencován touto proměnnou.

1.1 Lexikální jednotky

V SOL26 **záleží** na velikosti písmen u identifikátorů i klíčových slov (je *case-sensitive*).

- *Identifikátor* je neprázdná posloupnost číslic, písmen (malých i velkých) začínající malým písmenem; nebo případně *speciální identifikátor* `'_'` pro označení proměnné, která není dále použita. Jazyk SOL26 obsahuje navíc níže uvedená *klíčová slova*, která mají specifický význam, a proto se nesmějí vyskytovat jako identifikátory nových proměnných či jako selektory:

Klíčová slova: `class, self, super, nil, true, false`

- *Identifikátor třídy* má stejné podmínky jako *identifikátor*, ale začíná velkým písmenem. SOL26 také obsahuje vestavěné třídy (viz sekci 2) `Object, Nil, True, False, Integer, String, Block`.

Základním a jediným nositelem typu je třída. Pro jednoduchost není třeba pracovat s třídami jako s objekty první kategorie².

- *Pravdivostní literál* je vyjádřen klíčovým slovem `true` pro pravdu nebo klíčovým slovem `false` pro nepravdu. Tato klíčová slova se interpretují jako instance tříd `True` a `False` (viz sekci 2). Jde přitom o jedináčky (*singleton*), tj. tyto třídy mají v programu jediné instance zastoupené právě klíčovými slovy `true` a `false`.
- *Celočíselný literál* je tvořen neprázdnou posloupností číslic a vyjadřuje hodnotu celého čísla v desítkové soustavě s nepovinným unárním znaménkem (+ nebo -). Literál se interpretuje jako instance třídy `Integer` (klasický celočíselný rozsah implementačního jazyka interpretu).
- *Řetězcový literál* je oboustranně ohraničen jednoduchými uvozovkami (`' '`) a je reprezentován instancí třídy `String`. Tvoří jej libovolný počet znaků zapsaných mezi uvozovkami na jednom řádku. Možný je i prázdný řetězec (`' '`). Další znaky lze zapisovat pomocí escape sekvence: `'\ '`, `'\n'`, `'\\'` (význam je standardní).
- *Blokový literál* slouží pro zápis imperativního kódu. Tvoří jej *definice parametrů* a *sekvence příkazů* přiřazení uzavřená v hranatých závorkách. Obě části mohou být prázdné.

[*sekvence deklarací parametrů* | *sekvence příkazů přiřazení*]

Detailní popis syntaxe a sémantiky blokového literálu je v sekci 1.2.2.

- *Neplatná hodnota* je reprezentována klíčovým slovem `nil` (chová se obdobně jako pravdivostní literály – `nil` představuje jedinou instanci jedináčka `Nil`).
- *Blokový komentář* je ohraničen dvojími uvozovkami (`'"`).

1.2 Syntaxe a sémantika jazykových konstrukcí

Program v jazyce SOL26 se skládá ze sekvence definic tříd. Jedna z definovaných tříd musí být hlavní třída `Main` s bezparametrickou metodou `run` (příp. chyba 31). Interpret po spuštění auto-

¹Výjimkou jsou *interní instanční atributy*, které si pro instance objektů udržuje interpret, ale přímo z jazyka nejsou dostupné.

²To znamená, že třída samotná se v základním zadání nebere jako plnohodnotný objekt, tj. není možné ji např. referencovat z proměnné. Pro účely instanciac se však může vyskytnout na místě adresáta zprávy, kterým je jinak typicky instance třídy (objekt).

matically vytvoří instanci třídy `Main` a zašle jí zprávu (zavolá metodu) `run`.

Před, za i mezi jednotlivými tokeny se může vyskytovat libovolný počet bílých znaků, takže jednotlivé konstrukce jazyka SOL26 lze zapisovat na jednom či více řádcích. Za každým příkazem se píše tečka (`'.'`). Prázdný příkaz není dovolen. Na jednom řádku lze zapsat více příkazů.

Rozsáhlejší příklad programu je uveden níže v sekci 1.2.11.

1.2.1 Výrazy a selektory

V jazyce je k dispozici pouze jeden druh příkazu, a to *přiřazení* výsledku výrazu do proměnné. *Výrazem* je literál, proměnná, nebo zaslání zprávy. *Zaslání zprávy* se skládá z operandu (výraz, který se vyhodnotí na objekt, který bude příjemcem zprávy) a *selektoru* zprávy s případnými argumenty této zprávy (opět výrazy). Pomocí zasílání zpráv se realizuje jak invokace metod, tak přístup k instančním atributům objektů, a představuje tak primární způsob řízení toku programu.

Selektor zprávy slouží pro identifikaci zprávy při jejím zasílání (proloženo případnými argumenty) a pro definici jména metody reagující na zaslou zprávu, kterou instance třídy přijímá. Skládá se z jednoho či více identifikátorů dle toho, zda je bezparametrický, nebo s parametry. Identifikátory se mohou opakovat. Vyjma bezparametrického selektoru se bezprostředně za každým identifikátorem píše dvojtečka, počet dvojteček selektoru tak udává počet argumentů zprávy. Příklady selektorů:

run bezparametrický selektor.

plusOne: selektor s jedním parametrem.

startsWith:endsWith: selektor se dvěma parametry.

Při *zasílání* zprávy s daným selektorem se za každou dvojtečkou očekává výraz pro odpovídající argument zprávy. Příklady použití:

x := someExpr1 run. nepředává argumenty.

y := expr2 plusOne: 10. předává jako argument hodnotu 10.

z := e3 startsWith: 5 endsBefore: y. předává jako argumenty hodnotu 5 a hodnotu, kterou referencuje proměnná `y`.

1.2.2 Blok

Blok je objekt, který obsahuje sekvenci příkazů, kterou je možné vyhodnotit. Literál bloku:

```
[ sekvence deklarací parametrů bloku | sekvence příkazů přiřazení ]
```

je uzavřen v hranatých závorkách. První část obsahuje seznam parametrů, který může být i prázdný. Deklarace jednoho parametru se skládá z dvojtečky bezprostředně (bez bílých znaků) následované identifikátorem. Příklad:

```
[ | a := 1. |                               "Blok bez parametrů"
  [ :x|a := x. ]                             "Blok s jedním parametrem"
  [ :x :y :z | a := x plus: y. ]             "Blok se třemi parametry"
```

Následuje povinné svislítko `'|'` oddávající sekvenci příkazů přiřazení. Každý příkaz je ukončen tečkou. Proměnné uvnitř bloku nejsou dopředu deklarovány a jsou vytvářeny implicitně přiřazením objektu příkazem přiřazení. Parametry bloku se chovají jako nemodifikovatelné proměnné definované na začátku bloku.

1.2.3 Příkaz přiřazení

Příkaz *přiřazení* je jediným druhem příkazu v jazyce SOL26.

proměnná := výraz .

Všechny jeho části jsou povinné a je zakončen tečkou.

1.2.4 Proměnné a rozsah platnosti

Proměnné jazyka SOL26 jsou pouze lokální v blocích a lexikálně zanořených blocích (něco jiného jsou atributy objektů, viz sekci 1.2.10). Proměnná je definována prvním přiřazením v daném bloku a je viditelná i v lexikálně zanořených blocích, a to i v případě, že jsou takové bloky předány pomocí parametrů či instančních atributů jinam, kde mohou být později provedeny. SOL26 tedy podporuje *uzávěry* (angl. closures), v sekci 1.2.11 je uveden příklad. Při pokusu o přiřazení do formálního parametru dojde k chybě 34.

Při pokusu o *čtení* neinicilizované proměnné (např. ve výrazu) dojde k sémantické chybě 32.

Blok tvoří tzv. *rozsah platnosti*, ve kterém jsou dostupné uvnitř bloku definované proměnné. Lokální proměnné mají tedy rozsah platnosti od *místa jejich definice* až po konec bloku, kde byly definovány, včetně lexikálně zanořených bloků.³ V každém bloku jsou navíc k dispozici objekty **nil**, **true**, **false** s globální viditelností.

Tvoří-li blok tělo metody (tzv. *metodový blok*), obsahuje tento blok (i jeho podbloky) pseudoproměnné **self** a **super** (viz sekci 1.2.7).

1.2.5 Blok jako objekt

Blok je v SOL26 také objektem – instancí vestavěné třídy **Block** (viz také sekci 2). Je tedy možné mu zasílat zprávy (tj. invokovat jeho instanční metody), referencovat jej z proměnné, předávat jej jako argument ve zprávě. Příklad:

```
[ |
  "Blok bez parametrů přiřazený do 'b1'"
  b1 := [ | a := String read. _ := a print. ].
  "Blok s jedním par. přiřazený do 'b2'."
  S arg., který má rozumět zprávě plus: a po provedení by vrátil objekt pro x +
    1."
  b2 := [ :x | _ := x plus: 1. ].
  "Blok se dvěma par. přiřazený do 'b3'."
  Při vyhodnocení očekává v 'x' libovolný objekt,
  který umí zprávu 'value:' – tedy např. jiný blok."
  b3 := [ :x:y | val := x value: y. ].
  "Provedení bezparam. bloku = posláni zprávy 'value' objektu b1"
  _ := b1 value.
  "Provedení dvouparam. bloku zasláním zprávy 'value:value:' s argumenty
    'b2' a '3' objektu b3. V 'c' bude hodnota 4."
  c := b3 value: b2 value: 3.
]
```

Literál bloku při definici není hned vykonán! Uvedením literálu bloku na nějakém místě programu vznikne instance **Block**. Sekvenci příkazů uvnitř bloku je pak možné provést zasláním patřičné zprávy:

³Pozor, jde opravdu jen o *lexikálně* zanořené bloky. Pokud např. do metody přijde instance *b* jiného bloku jako argument, při provedení tohoto bloku *b* nejsou dostupné lokální proměnné metodového bloku, ze kterého byl *b* spuštěn.

- pokud je literálem bloku definovaná instanční metoda ve třídě (jde o *metodový blok*), provede se obsah bloku zasláním zprávy s příslušným selektorem instanci této třídy (viz sekci 1.2.10);
- pokud podle literálu bloku vznikl objekt, který je referencovaný z nějaké proměnné, provede se obsah bloku zasláním zprávy **value** (příp. **value:**, **value:value:** atd. podle počtu parametrů bloku; zaslání zprávy se špatnou aritou vede na chybu 51) tomuto objektu.

1.2.6 Výsledek bloku

Výsledkem provedení bloku je hodnota posledního vyhodnoceného výrazu v posledním provedeném příkazu bloku. Pokud žádný výraz vyhodnocen nebyl, výsledkem je **nil**.

```
class Main : Object {
  run [|
    a := self foo: 4. "a = instance 14"
    b := [ :x | _ := 42. ]. "b = instance Block"
    c := b value: 16. "c = instance 42"
    d := 'ahoj' print.] "d = instance 'ahoj' - print vrátí self, viz Vestavěné třídy"
  foo: [ :x |
    "s proměnnou 'u' se nijak dál nepracuje, ale výsledek zaslání zprávy 'plus:' bude vrácen jako výsledek volání metody 'foo'"
    u := x plus: 10.
  ]
}
```

1.2.7 Reference na příjemce zprávy self

Při invokaci metody je prostředí odpovídajícího *metodového bloku* doplněno o předinicializovanou nemodifikovatelnou (pseudo)proměnnou **self**, která referencuje objekt, který zprávu přijal. V lexikálně zanořených blocích je **self** (a **super**) z metodového bloku viditelné a dostupné stejně jako ostatní proměnné.

Uvažujme složitější příklad níže, který demonstruje *statický rozsah platnosti* pseudoproměnné **self**. Pseudoproměnná **self** je uvnitř bloku navázána v době *definice* bloku na instanci, která je kontextovým objektem provádění metody, která tento blok obsahuje, a to i zanořeně. Blok si tuto referenci pro **self** pamatuje po zbytek své existence a při provádění bloku je tato reference využita (i když je blok prováděn třeba v kontextu jiného objektu).

```
class Main : Object {
  run [|
    a := A new.
    "Instance bloku si zapamatuje, že self odkazuje na tuto instanci Main. Blok navíc tuto referenci na konci vrátí."
    b := [ :arg | y := self attr: arg. z := self. ].
    "Zavoláme metodu 'foo' na instanci A a předáme jí objekt 'b' typu Block."
    c := a foo: b.
    "Výsledkem přiřazeným do c je instance třídy Main s instančním atributem attr inicializovaným na 'foo' - můžeme vypsát."
    _ := (self attr) print.
  ]
}

class A : Object {
  foo: [ :x |
    "Blok předaný v x je vyhodnocen a do instance Main je jím vytvořen instanční atribut attr s hodnotou 'foo'."
    u := x value: 'foo'. ]
}
```

1.2.8 Klíčové slovo **super**

Dále je dostupné klíčové slovo **super**, které odkazuje na stejný objekt jako **self**, ale při použití na místě příjemce zprávy upravuje, kde se bude hledat metoda odpovídající selektoru. Metoda bude nejprve vyhledávána až v přímé nadtřídě třídy, v jejíž metodě je zaslání zprávy realizováno, a pak postupně hledá v dalších nadtřídách. Díky tomu je možné vynutit provedení původní metody v případě její redefinice.

Pokud je **super** někde přiřazeno nebo předáno jako argument, chová se jako synonymum pro **self**.

```
class A : Object {
  m: [ :x | _ := x print. ]
  r [      | _ := self print. ]
}

class B : A { "B dědí z A, redefinuje metodu m"
  m: [ :x | _ := super m: 'ahoj'. "super -> použije se implementace z A"
      _ := x print. ]
}

class C : B {
  "self m najde metodu m z nadtřídě B a pošle jí jako argument instanci C (na
  místě argumentu má super stejný význam jako self)."
  u [ | _ := self m: super. ]
  print [ | _ := 'bar' print. ]
}

class Main : Object {
  run [ | c := C new. "Vytvoří instanci c třídy C."
      "Této instanci pošle zprávu m -> implementace se najde v B. m (z B)
      nejprve zavolá implementaci z A, což vypíše 'ahoj'. Pak m (z B)
      zavolá na argumentu print, takže se vypíše 'foo'."
      _ := c m: 'foo'.
      "Instanci c se pošle zpráva u, odpovídající metoda zavolá m (najde se v
      B) s argumentem super (= self). Ta pak volá m (z A), která pošle
      svému argumentu zprávu print. Metoda pro tuto zprávu se najde v C,
      protože instance C je tím objektem, který se poslal jako argument x."
      _ := c u.
      "Instanci c pošle zprávu r. Implementace se najde v A. Tato metoda
      posílá do self zprávu print. Protože self je v tomto kontextu
      instancí C, instance použije svou metodu print (definována v C) a
      vypíše se 'bar'."
      _ := c r.
  ]
}
```

1.2.9 Výrazy a zaslání zpráv

Výrazem je literál, proměnná, nebo zaslání zprávy. Výsledkem výrazu je vždy objekt. Literálem může být číslo, řetězec, identifikátor třídy nebo blok. Výrazem je také proměnná včetně **true**, **false**, **nil**, **self**, **super**.

Každé zaslání zprávy (mimo takové, které na nejvyšší úrovni leží bezprostředně za ':=') musí být uzavřeno v závorkách. Formálněji řečeno, zaslání zprávy není asociativní a všechny zprávy (bezparametrické i s parametry) mají stejnou prioritu.

V případě klasického zaslání zprávy je na místě příjemce zprávy **výraz**, který se vyhodnotí na objekt; zpráva je pak určena selektorem. Příklad příkazu přiřazení s výrazem zaslání zprávy se

dvěma argumenty:

```
x := targetExpression selector: arg1Expression arg: arg2Expression.
```

Jednotlivými argumenty jsou opět výrazy – při zaslání zprávy se tyto výrazy nejprve vyhodnotí zleva doprava⁴ a reference na výsledné objekty se použijí jako argumenty zprávy (tj. předají se do těla invokovaného bloku).

```
class Main : Object {
  run [|
    "výraz zaslání zprávy v argumentu je už nutné uzávkovat"
    a := self attrib: (Integer from: 10).
    "vnořená zaslání zprávy, kde je výsledek užit jako cíl pro další zprávu"
    b := [| x := ((self attrib) asString) concatenateWith: (10 asString). |].
  ]
}
```

Vestavěné třídy nabízejí několik vestavěných *třídních metod* v roli konstruktoru (např. **new** nebo **from:**), které jsou invokovány zasláním *třídní zprávy*, kdy je příjemce vyjádřen literálem pro identifikátor třídy:

```
x := String from: 'Test\n'.
```

Není možné vytvářet vlastní (uživatelské) třídní metody. Pokud třída nerozumí zaslané třídní zprávě, vede to na statickou chybu 32.

1.2.10 Třídy, instanční metody a atributy

Definice třídy se skládá z klíčového slova **class** následovaného identifikátorem třídy, dvojtečkou a identifikátorem nadtřídy (angl. *superclass*). Uvedení nadtřídy je povinné (nemůže se tak stát, že by předkem třídy nebyl **Object**). Tělo třídy je ohraničeno závkami '{ }', uvnitř těla se nachází žádná či více instančních metod. Invokací konstrukturu vzniká instance třídy (čili konkrétní objekt). Té je možné zasílat zprávy, kterým instance *rozumí*, tj. vlastní třída nebo nadtřída metodu reagující na zprávu definuje. Pokud zaslanou zprávu třída nepodporuje (tj. neexistuje metoda nebo instanční atribut odpovídající selektoru), program končí s chybou 51, že *instance nerozuměla zprávě*.

Všechny identifikátory tříd (vestavěné i uživatelem definované) mají globální viditelnost.

Instanční metody

Definice instančních metod jsou v těle třídy ve tvaru:

```
selektor [ "blokový literál s odpovídajícím počtem parametrů" ]
```

Blokový literál uvedený za selektorem reprezentuje tělo metody (nesouhlasí-li *arita* selektoru a bloku, dojde k sémantické chybě 33). Při zaslání zprávy instanci nejprve dojde k vyhledání odpovídající instanční metody (dle souhlasného selektoru a stejné arity) ve vlastní třídě příjemce. V invokované metodě je navázána pseudoproměnná **self** na příjemce zprávy a jednotlivým parametrům jsou přiřazeny hodnoty argumentů (výrazy argumentů jsou vyhodnoceny postupně zleva doprava). Jako výsledek metody je navrácen výsledek bloku reprezentujícího tělo invokované metody.

Instanční metody jsou do třídy zděděny z jejích předků. Zděděnou instanční metodu lze v potomkovi redefinovat (angl. *override*). Při přijetí zprávy se metoda hledá nejprve ve vlastní třídě příjemce, pak postupně v jeho předcích.

⁴Povšimněte si, že je použita vyhodnocovací strategie *eager evaluation*.

Přetěžování (angl. *overloading*), tj. použití stejného selektoru pro více různých implementací metod, není v SOL26 podporováno.

Instanční atributy

Uživatelsky definovaný *instanční atribut* není deklarován ve třídě⁵, ale je vytvářen a inicializován programově pomocí **jednoparametrického selektoru**⁶. Výsledkem inicializační/nastavovací zprávy instančního atributu je **self** (tj. odkaz na instanci s modifikovanou hodnotou onoho atributu).

Přístup k hodnotě dříve definovaného instančního atributu provádíme bezparametrickou zprávou se selektorem odpovídajícím identifikátoru atributu. Nelze vytvořit instanční atribut, jehož čtení⁷ by se krylo s existující bezparametrickou instanční metodou (vč. zděděných) příjemce (jinak chyba 54).

```
class Main : Object {
  run [|
    "definuje a inicializuje instanční atribut 'value'"
    r := self value: 10.
    "definuje další inst. atribut 'next', inicializuje hodnotou atributu 'value'"
    e := self next: (self value).
    "atribut 'value' již existuje, takže pouze modifikuje hodnotu na nil"
    t := self value: nil.
    "CHYBA 54! Třída definuje metodu foo, jméno nelze použít jako atribut."
    _ := self foo: 10.
  ]

  foo [|]
}
```

Protože jsou instanční atributy vázány na konkrétní objekt (instanci), nemá smysl (v kontextu dědičnosti) uvažovat u nich pojem redefinice (na rozdíl od metod). Pro účely nastavování instančních atributů a přístupu k nim se **self** a **super** chovají obvykle jako synonyma – výjimkou je právě kontrola kolizí s definovanými metodami, která při použití **super** uvažuje jen metody definované na nadtřídách, zatímco při použití **self** uvažuje i metody definované vlastní třídou příjemce. Může se tedy např. stát, že třída definuje instanční metodu, která překryje instanční atribut vzniklý např. v nějaké metodě nadtřídy. V tomto případě lze k instančnímu atributu přistoupit pomocí **super**.

```
class Parent : Object {
  something [|
    "definuje a inicializuje instanční atributy 'p' a 'q'"
    p := self p: 'hello'.
    q := self q: 'world'.
    "Je-li příjemce instance Parent, získá hodnotu v 'p' a vytiskne 'hello'."
    "Je-li příjemce instance Main, reaguje invokací metody 'p' a vypíše 'foo'."
    _ := (self p) print. ] }
class Main : Parent {
  run [|
    "tyto čtyři příkazy mají stejný efekt"
    a := super value: super. b := super value: self.
    c := self value: super. d := self value: self.

    x := self p: 'val'. "CHYBA 54! Vedlo by na vznik atributu, ale už máme
      instanční metodu p, takže atribut nelze vytvořit."
```

⁵Podobně jako v jazyce Python 3.

⁶Instanční atribut tedy vznikne, pokud zašleme zprávu s jedním parametrem, které příjemce zatím „nerozumí“.

⁷Z podstaty věci nelze vytvořit ani instanční atribut, pro který by vznikla kolize se zapisovací zprávou (jeho vznik by se provedl posláním takové zprávy, na kterou už příjemce umí odpovědět).


```

x := self p. "hodnotou x bude 'foo'"

x := super something: 1. "CHYBA 54! Vedlo by na vznik atributu, ale v
    nadtřídě je kolizní metoda."
x := super p. "CHYBA 51! Nadtřída nezná selektor p. (something se zatím
    neprovedla)"

_ := super something. "nastaví atributy p, q a vypíše 'foo'"
x := self p. "x = 'foo' (volání metody)"
y := super p. "y = 'hello' (přístup k atributu)"
z := self q. "q = 'world' (přístup k atributu)"
]

p [|
    "instanční metoda, která jen vrátí řetězec 'foo'"
    _ := 'foo'.
]
}

```

Všechny instanční atributy (vyjma interních) jsou veřejné (angl. *public*). Přístup k zatím nedefinovanému instančnímu atributu vede na interpretační chybu 51.

Především z pohledu implementace obsahují objekty také *interní instanční atributy*. Jde např. o skutečnou numerickou hodnotu objektu, který reprezentuje celé číslo, nebo o identifikátor objektu. Tyto atributy objektů jsou užity pouze pro interní účely interpretace, jsou naplněny během instanciaci a uživatelsky (z kódu SOL26) nejsou přístupné ani modifikovatelné.

Konstruktory

Za účelem vytváření instancí tříd existují v jazyce také dvě *třídní* metody, které fungují jako konstruktory. Formálně jsou tyto metody definovány na třídě **Object**, díky dědičnosti jsou pak i ve všech jejích podtřídách. Chování třídních metod není možné uživatelsky měnit, metody nelze redefinovat (angl. *override*). Vestavěná třída **String** má nad rámec následujících dvou konstruktorů ještě další třídní metodu (konstruktor) **read** (viz sekci 2).

new bezparametrická třídní metoda, která vytvoří novou instanci dle třídy příjemce a případně inicializuje interní instanční atributy implicitními hodnotami (viz sekci 2).

from: třídní metoda s jedním parametrem (zde označený za *obj*), která vytvoří novou instanci třídy, která tuto třídní zprávu přijala, a inicializuje:

- interní instanční atributy kopií hodnot těchto atributů z *obj*,
- instanční atributy tzv. mělkou kopií hodnot všech instančních atributů z *obj* (to znamená, že vytvořená instance bude mít všechny instanční atributy, které má *obj*, a ty budou referencovat stejné objekty, které jsou referencovány z *obj*).

Pokud definuje přijímající třída interní instanční atribut, je nutné, aby jej měl k dispozici i objekt *obj* (jinak chyba 53).

Příklad použití **from**:

```

class Factorial : Integer {
    factorial "použití from: pro podtřidu třídy Integer"
    [| r := (self equalTo: 0) ifTrue: [|r := Factorial from: 1.]
        ifFalse: [|r := self multiplyBy:
            ((Factorial from: (self plus: -1)) factorial). |.]
    ]
}

```

```

class Main : Object {
  run
  [| x := Factorial from: ((String read) asInteger).
   x := ((x factorial) asString) print.

   "CHYBA 53! Factorial dědí z Integer, tudíž obsahuje interní instanční
   atribut s číselnou hodnotou - řetězec takový interní atribut nemá"
   y := Factorial from: 'str'.
  ]
}

```

1.2.11 Příklady

Demonstrace metod, selektorů, základního zasílání zpráv

```

class Main : Object {
  run "<- definice metody - bezparametrický selektor run"
  [ |
    "Zašle zprávu 'compute:and:and:' sobě samému - selektor se třemi arg."
    x := self compute: 3 and: 2 and: 5.
    "Nejprve zašle zprávu 'vysl' objektu self (bezparametrický selektor). Pak
    zašle zprávu 'plusOne:' (selektor s jedním parametrem) sobě samému -
    hodnotou argumentu je výsledek předchozí zprávy."
    x := self plusOne: (self vysl).
    "Zašle zprávu 'asString' objektu x - bezparametrický selektor."
    y := x asString.
  ]

  plusOne: "<- definice metody - selektor s jedním parametrem"
  [ :x | r := x plus: 1. ]

  compute:and:and: "<- definice metody - selektor se třemi parametry"
  [ :x :y :z |
    "Zašle zprávu 'plus:' objektu x - selektor s jedním parametrem."
    a := x plus: y.
    "Zašle zprávu 'vysl:' sobě samému - nastaví instanční atribut 'vysl'."
    _ := self vysl: a.
    "Pošle si zprávu 'vysl', výsledkem je ref. na objekt referencovaný
    atributem vysl; tomuto objektu pak zašle zprávu 'greaterThan:', jejím
    argumentem je 0."
    _ := ((self vysl) greaterThan: 0)
    "Výsledkem je objekt typu True nebo False, kterému se zašle zpráva
    'ifTrue:ifFalse:', argumenty jsou bezparametrické bloky."
    ifTrue: [|u := self vysl: 1.]
    ifFalse: [|].
  ]
}

```

Povšimněte si, že identifikátor se může v selektoru opakovat. Selektor vyjadřuje pouze „rozhraní“ zprávy – předané argumenty jsou vázány na parametry bloku podle deklarovaného pořadí, ne podle identifikátoru v selektoru.

Demonstrace uzávěrů

Předpokládejte, že na standardním vstupu programu se nachází 1\n2\n3\n4\n.

```
class Main : Object {
  giveObjectWithBlock [ |
    "Načte řádek a vytvoří objekt pro číslo 1."
    x := ((String read) asInteger).
    "Vytvoří blok, který lze opakovaně vyhodnotit. Součástí bloku je v uzávěru
      odkaz na lokální proměnnou x."
    b := [| y := ((String read) asInteger) plus: x. |].
    "Vytvoří instanci třídy Object a v ní rovnou vytvoří instanční atribut
      block, do kterého nastaví výše vytvořený blok."
    x := 7.
    z := ((Object new) block: b).
    "Projeví se i v uzávěru bloku."
    x := 9.
    "Zde metoda končí, ale proměnná x stále „žije“ v uzávěru bloku b.
      Následující příkaz způsobí, že giveObjectWithBlock vrátí vytvořený
      objekt z jako svou návratovou hodnotu."
    _ := z.
  ]

  run "vstupní bod programu"
  [ |
    "Vytvoří objekt s blokem."
    o := self giveObjectWithBlock.
    "Načte řetězec '2' (první načtení již proběhlo)."
    _ := String read.
    "V rámci vyhodnocení bloku načte číslo 3, vrátí 12 (9 + 3)."
    r1 := ((o block) value).
    "V rámci vyhodnocení bloku načte číslo 4, vrátí 13 (9 + 4)."
    r2 := ((o block) value).
  ]
}
```

2 Vestavěné třídy SOL26

Vestavěné instanční metody, které vyžadují jako argument instanci, která rozumí zprávě **value** či **value:** (např. bezparametrický blok a blok s jedním parametrem), a nedostanou ji, způsobí chybu 51.

Třída **Object**

Instanční metody:

identicalTo: Testuje shodu dvou objektů, tj. že se jedná o *tentýž* objekt.

equalTo: Datově porovná objekt: pokud objekt nemá interní atributy, invokuje **identicalTo:**, jinak porovnává interní atributy (v potomcích je možné **equalTo:** redefinovat tak, aby vhodně porovnávala i instanční atributy).

asString Vrací řetězec '' (v potomcích redefinováno rozumnější implementací).

isNumber, isString, isBlock, isNil Vrací **false**. V potomcích jsou redefinovány tak, že vrací **true**, pokud je příjemce instance **Integer / String / Block / Nil** (nebo jejich podtřídy).

isBoolean Vrací **false**. V potomcích **True** a **False** je redefinována tak, že vrací **true**.

Třída **Nil : Object**

Třída **Nil** je implementována jako jedináček. Konstruktory **new** i **from:** vždy vrací stejnou instanci **Nil**, která je totožná s globálně dostupným objektem **nil**⁸. Třídní zprávy **new** a **from:** se pro případné podtřídy **Nil** chovají jako jiné běžné uživatelsky definované třídy (již nezajišťují jedináčka).

Instanční metody:

asString Vrací řetězec '**nil**'.

Třída **Integer : Object**

Instance této třídy zastupují celá čísla. Odpovídající celé číslo je uloženo v interním instančním atributu. Výchozí hodnota (při použití **Integer new**) je 0.

Použitím celočíselného literálu vzniká instance třídy **Integer**, přičemž není definováno, zda jde pro konkrétní literál vždy o stejnou instanci. Je tedy možné (ale ne nutné), aby byla identita nějaké instance **Integer** nebo její podtřídy vztažena právě k internímu atributu s hodnotou – konstruktor **from:** může vrátit existující instanci. Příklad:

```
class MyInt : Integer { }
class Main : Object {
  run [|
    x := 1. y := 1. z := Integer from: 1.
    u := MyInt from: 1. w := MyInt from: 1.

    a := x equalTo: y. a := x equalTo: z.  "obojí true"
    a := x identicalTo: y.  "podle implementace"
    a := x identicalTo: z.  "podle implementace"

    a := u equalTo: x. a := u equalTo: w.  "obojí true"
    a := u identicalTo: x.  "false"
```

⁸Není tedy možné získat dvě instance **Nil**, pro které by **identicalTo:** vrátila **false**.

```

    a := u identicalTo: w. "podle implementace"
  ]
}

```

Instanční metody:

equalTo: Porovná, zda je číselná hodnota (v interním instančním atributu) příjemce a argumentu shodná.

greaterThan:, plus:, minus:, multiplyBy: Standardní numerické operace.

divBy: Celočíselné dělení kompatibilní s implementačním jazykem interpretu⁹. Dělení nulou vede na chybu 53.

asString Vrací číslo převedené na řetězec v desítkové reprezentaci a s případným znaménkem mínus (kladné znaménko se vynechává).

asInteger Vrací sebe sama.

timesRepeat: Jako argument očekává instanci, která rozumí zprávě **value:**¹⁰. Pokud (a jen tehdy, když) je příjemce $n > 0$, blok z argumentu se provede n -krát. Bloku resp. argumentu se předá jako argument číslo iterace (od 1 do n včetně). Návratovou hodnotu udává návratová hodnota posledního vykonání bloku z argumentu (nebo **nil**, pokud se žádný nevykoná).

Třída **String** : **Object**

Instance této třídy reprezentují řetězce. Řetězec je uložen v interním instančním atributu. Výchozí hodnota (při použití **String new**) je prázdný řetězec **' '**. Použitím řetězcového literálu vzniká instance třídy **String**, přičemž není definováno, zda jde pro konkrétní literál vždy o stejnou instanci.

Třídní metody:

read Načte řetězec z jednoho řádku vstupu (tj. po odřádkování včetně, které ale není součástí načteného řetězce) a vytvoří odpovídající instanci **String**.

Instanční metody:

print Vytiskne řetězec na výstup (bez jakýchkoliv formátovacích znaků), vrací **self**.

equalTo: Porovná, zda je řetězec (v interním inst. atributu) příjemce a argumentu shodný (odpovídá běžnému porovnání shody řetězců v implementačním jazyce interpretu).

asString Vrací sebe sama.

asInteger Pokud lze příjemce jednoduše převést na celé číslo, vrací instanci **Integer**, jinak vrací **nil**.

concatenateWith: Vrací novou instanci **String**, která obsahuje konkatenaci příjemce a argumentu (argument musí být instance třídy **String** nebo její podtřídy, jinak vrací **nil**).

startsWith:endsBefore: Vrací podřetězec od indexu daného prvním argumentem (indexuje se od 1) po předchozí znak daným druhým argumentem. Je-li rozdíl argumentů menší či roven 0, vrací prázdný řetězec. Nejsou-li argumenty kladná nenulová celá čísla, vrací **nil**. Je-li **endsBefore** větší než délka řetězce, vrací podřetězec až po konec příjemce.

⁹<https://www.php.net/manual/en/function.intdiv.php>

¹⁰Např. blok s jedním parametrem.

length Vrací počet znaků řetězce (jako instanci **Integer**). Jedna escape sekvence se počítá jako jeden znak.

Třída **Block** : **Object**

Instance této třídy reprezentují bloky kódu. Pokud instance vzniká podle blokového literálu, interpret zajistí, že tato instance obslouží zprávu s identifikátorem **value**, která očekává příslušný počet parametrů. Bezparametrický blok tak bude obsahovat metodu **value**, zatímco např. blok se dvěma parametry bude obsahovat metodu **value: value: .** Třídní zpráva **new** zasláná třídě **Block** vytvoří novou instanci reprezentující prázdný bezparametrický blok (včetně bezparametrické metody **value**).

Další instanční metody:

whileTrue: Jako argument přijímá instanci (tzv. tělo), která rozumí zprávě **value** (např. bezparametrický blok), který se opakovaně provádí, dokud je příjemce vyhodnocen jako **true**. Návrátovou hodnotou je návratová hodnota naposledy provedeného těla, nebo, nebylo-li tělo provedeno ani jednou, tak **nil**.

```
"tělo nějaké metody v rámci nějaké třídy, kde se vytvoří atribut 'attr'"
x := self attr: 3.
y := [| ret := (self attr) greaterThan: 0. ] whileTrue:
    [| r := ((self attr) asString) print.
     r := self attr: ((self attr) minus: 1). ]
```

Třídy **True** : **Object** a **False** : **Object**

Třídy **True** a **False** reprezentují logické hodnoty pravdy a nepravdy. Každá z těchto tříd je implementována jako jedináček přístupný přes globálně viditelný objekt **true** a **false** (obdobně jako **Nil**).

Instanční metody:

asString Pro objekty **true** a **false** vrací řetězec '**true**' nebo '**false**'.

not Vrací negaci logické hodnoty příjemce.

and: Je-li příjemce **false**, vrací **false**. Je-li příjemce **true**, vyhodnotí se argument zasláním zprávy **value** (tj. např. bezparametrický blok).

or: Je-li příjemce **true**, vrací **true**. Je-li příjemce **false**, vyhodnotí se argument zasláním zprávy **value** (argument je např. bezparametrický blok).

ifTrue:ifFalse: Je-li příjemce **true**, vyhodnotí se první argument zasláním zprávy **value**. Je-li příjemce **false**, vyhodnotí se druhý argument zasláním zprávy **value**. Návratovou hodnotou je návratová hodnota provedeného bloku.

isBoolean Vrací **true**.

Těmito metodami je tedy podporováno zkrácené vyhodnocování pravdivostních výrazů. Dokud argument není vyhodnocen zasláním zprávy, tak nemusí vadit, že dané zprávě nerozumí.

```
"Příklad výrazu s pravdivostními hodnotami"
y := (true and: [| _ := false isBoolean. ]) or: [| _ := nil error].
"vrací true, blok s nil error není vůbec vykonán"
_ := (y asString) print.
```

Poznámky k implementaci

V případě nekompletní implementace se nejprve zaměřte na možnost definice uživatelské třídy **Main** s metodou **run** a možnost provádět příkazy v bezparametrickém bloku, dále podporujte výstupní metody a minimalisticky implementované vestavěné třídy **Object**, **String** a **Integer**.

3 Přehled chybových návratových kódů

Jestliže proběhne činnost skriptu bez chyb, vrací se návratová hodnota 0 (nula). Jestliže dojde k nějaké chybě, vrací se chybová návratová hodnota podle přehledu níže.

Pozor: Veškerá chybová hlášení, varování a ladicí výpisy směřujte pouze na standardní **chybový** výstup (`stderr`), jinak vaše řešení neprojde automatickými testy!

- 10 chybí parametr příkazové řádky, který je povinný, nebo je použita zakázaná kombinace parametrů
- 11 chyba při otevírání vstupních souborů (např. neexistence, nedostatečné oprávnění)
- 20 chybný XML formát ve vstupním souboru (soubor není tzv. dobře formátovaný, angl. *well-formed*, viz [1])
- 42 neočekávaná struktura XML (např. špatné zanoření elementů, špatná hodnota atributu `language`, chybějící povinné atributy, špatné hodnoty atributu `order`, apod.)
- 31 statická sémantická chyba – chybějící třída **Main** či její instanční metoda **run**
- 32 statická sémantická chyba – použití nedefinované (a tedy i neinicializované) proměnné, formálního parametru, třídy, nebo třídní metody
- 33 statická sémantická chyba arity – špatná arita bloku přiřazeného k selektoru při definici instanční metody
- 34 statická sémantická chyba – pokus o přiřazení do formálního parametru bloku (tj. formální parametr na levé straně příkazu přiřazení)
- 35 jiná statická sémantická chyba (např. redefinice třídy, kolizní jména parametrů ve stejném bloku a podobně)
- 51 běhová chyba interpretace – příjemce nerozumí zaslané zprávě (a nejde o vytváření instančního atributu)
- 52 běhová chyba interpretace – ostatní chyby
- 53 běhová chyba interpretace – špatná hodnota argumentu (např. dělení nulou, nekompatibilní argument pro třídní zprávu **from:**)
- 54 běhová chyba interpretace – pokus o vytvoření instančního atributu, jehož název by kolidoval s metodou
- 99 neočekávaná interní chyba (neovlivněná integrací, vstupními soubory či parametry příkazové řádky)

Chyby 31–35 jsou označeny jako „statické“, neboť je lze detekovat prostou statickou analýzou vstupního programu bez jeho provádění. Chyby 51–54 jsou označeny jako „běhové“, neboť je není možné spolehlivě detekovat bez provádění kódu. Vaše interpretace **nemusí** (ale může) provádět statickou analýzu – i statické chyby mohou být detekovány až při běhu, tudíž pouze v případech, kdy se na ně při konkrétním spuštění narazí.

4 Popis XML reprezentace programu

Za povinnou XML hlavičkou¹¹ následuje kořenový element **program** s povinným textovým atributem `language` s hodnotou `"SOL26"` a případně nepovinným textovým atributem `description` s krátkým popisem programu.

Element `program` obsahuje pro uživatelsky definované třídy podelementy **class**. Každý element `class` obsahuje dva povinné atributy:

- `name` s identifikátorem třídy,
- `parent` s identifikátorem nadtřídy (rodiče).

Element `class` pak obsahuje podelement **method** pro každou definovanou instanční metodu, a to s povinným atributem `selector` pro určení selektoru definované metody. Element `method` potom obsahuje element pro blok s odpovídajícím počtem argumentů.

U příkazů v bloku, parametrů a argumentů je důležité pořadí. Protože XML nezajišťuje u elementů na stejné úrovni pořadí, je nutné u těchto elementů pořadí explicitně zapsat použitím povinného textového atributu `order`, který je vždy číslován souvisle a od jedničky (viz příklad níže).

Blok je reprezentován elementem **block** s povinným atributem `arity` udávajícím počet očekávaných argumentů pro budoucí vyhodnocení bloku. Element `block` obsahuje podelementy **parameter** pro každý parametr bloku se dvěma povinnými atributy `order` a `name` pro pořadí a identifikátor parametru. Dále element `block` obsahuje podelementy pro každý příkaz sekvence příkazů.

Příkaz je reprezentován elementem **assign** s povinným atributem `order` pro určení pořadí příkazu v sekvenci příkazů. Příkaz zahrnuje dva povinné podelementy **var** s atributem `name` pro identifikátor cílové proměnné a podelement `expr` pro výraz pro výpočet přiřazované hodnoty.

Výraz (element **expr**) obsahuje jeden podelement podle druhu výrazu:

- literál (`literal`, viz dále),
- proměnná (`var`, viz výše),
- blokový literál (`block`, viz výše) nebo
- zaslání zprávy (`send`, viz dále).

Element **literal** obsahuje dva povinné textové atributy `class` s identifikátorem vestavěné třídy (`Integer/String/Nil/True/False`) a atribut `value` reprezentující hodnotu literálu (primitivní), např. číslo `-10`, prázdný řetězec, či `true`.

U literálů typu `String` jsou v XML převedeny původní escape sekvence na odpovídající XML entity (např. `\n` na `
`). XML entity jsou využity také pro znaky, které v XML nelze jinak zapsat (`<`, `>`, `&`, `'`, `"` jsou převedeny na `<`, `>`, `&`, `'`, `"`).

Pro vyjádření literálu identifikátoru třídy je atribut `class` nastaven na `"class"` a `value` obsahuje identifikátor třídy. Literál bloku je reprezentován elementem `block` (viz výše).

Zaslání zprávy je reprezentováno elementem **send**, jehož selektor zprávy je uložen v povinném textovém atributu `selector`. Výraz pro vyhodnocení příjemce zprávy je v podelementu `expr` (viz výše). Pokud se jedná o zprávu s parametry, obsahuje element `send` ještě podelement **arg** pro každý argument předávaný zprávě. Element `arg` má povinný argument `order` a obsahuje právě jeden podelement `expr` (viz výše) pro výraz, jehož vyhodnocením získáme skutečný argument zprávy.

¹¹Tradiční XML hlavička včetně verze a kódování je `<?xml version="1.0" encoding="UTF-8" ?>`

Příklad úryvku jazyka SOL26:

```
[ :one :two | r := Integer from: two. ]
```

a jemu odpovídající XML:

```
<block arity="2">
  <parameter name="one" order="1" />
  <parameter name="two" order="2"></parameter>
  <assign order="1">
    <var name="r" />
    <expr>
      <send selector="from:">
        <expr><literal class="class" value="Integer" /></expr>
        <arg order="1">
          <expr><var name="two" /></expr>
        </arg>
      </send>
    </expr>
  </assign>
</block>
```

Reference

- [1] Extensible Markup Language (XML) 1.0. W3C. World Wide Web Consortium [online]. 5. vydání. 26. 11. 2008 [cit. 2020-02-03]. Dostupné z: <https://www.w3.org/TR/xml/>

Revize specifikace

2026-02-26: Oprava špatné závorky v příkladu na str. 4.

2026-02-27: Oprava příkladu na str. 11.

2026-03-06: Oprava komentáře v prvním příkladu v sekci 1.2.11. Oprava syntaxe v příkladu v sekci 1.2.8. Oprava překlepu ve větě v sekci **Konstruktory**.